
1.1 ¿Qué es Dart y por qué Flutter lo eligió?

Dart es un lenguaje cliente optimizado, desarrollado por Google, orientado a objetos, con tipado estático y compilado. Nació originalmente con la idea de sustituir o complementar a JavaScript en los navegadores, pero encontró su verdadera "killer app" cuando el equipo de Flutter lo seleccionó como su motor primario.

Como ingenieros en sistemas, deben preguntarse: *¿Por qué Dart y no Kotlin, Swift o Java?* La respuesta radica en cómo gestiona la compilación y la ejecución:

- **Just-In-Time (JIT) Compilation:** Durante la fase de desarrollo, Dart compila el código sobre la marcha. Esto permite la funcionalidad estrella de Flutter: **Stateful Hot Reload**. Cambias una línea de código en VS Code, guardas (**Ctrl + S**), y en menos de un segundo ves el cambio reflejado en el emulador sin perder el estado de la aplicación.
- **Ahead-Of-Time (AOT) Compilation:** Al generar el binario para producción (**release**), Dart compila directamente a código máquina nativo (ARM64, x86_64). No hay puentes (*bridges*) de JavaScript ni entornos de ejecución pesados intermediando entre tu app y el procesador del dispositivo Android. Esto nos da un rendimiento sostenido de 60 a 120 FPS.
- **Garbage Collection Eficiente:** El recolector de basura de Dart está optimizado para crear y destruir objetos de vida corta a una velocidad increíble, encajando a la perfección con la naturaleza reactiva de Flutter, donde los *widgets* se reconstruyen constantemente.

1.2 Configuración del Entorno en VS Code

Para trabajar de manera adecuada en este curso, nos aseguraremos de tener la última versión estable de **Dart SDK** (incluida al instalar Flutter SDK).

Extensiones obligatorias para VS Code:

1. **Flutter** (desarrollada por Dart Code): Incluye automáticamente la extensión de **Dart**.
2. **Error Lens:** Muestra los errores sintácticos y advertencias en la misma línea de código.
3. **Bracket Pair Colorization Toggler** (o la función nativa activada): Crucial para no perderse entre paréntesis y corchetes anidados.

Pueden verificar que todo esté en orden abriendo su terminal integrada en VS Code (**Ctrl + ~**) y ejecutando:

```
dart --version
flutter doctor
```

1.3 Conceptos Fundamentales del Lenguaje

Antes de escribir nuestro primer script, repasemos los pilares sintácticos y conceptuales de Dart.

Sound Null Safety (Seguridad Nula Sólida)

A partir de Dart 2.12, el lenguaje adoptó *Sound Null Safety*. Esto significa que las variables son **no-nulas por defecto**. El compilador detecta errores de valor nulo en tiempo de compilación y no en tiempo de ejecución (evitando el famoso `NullPointerException`).

- `String nombre`: Nunca puede valer `null`.
- `String? apellido`: Puede contener un `String` o un valor `null` gracias al operador `?`.

Tipado Estático e Inferencia de Tipos

Dart detecta tipos en tiempo de compilación. Puedes declarar variables asignando explícitamente el tipo de dato o usando la palabra reservada `var` para que el compilador infiera el tipo según el valor asignado. Una vez inferido, el tipo no puede cambiar.

Tipos de Datos Primarias

- `int`, `double` (ambos heredan de `num`).
- `String` (soporta interpolación nativa con `$variable` o `${expresion}`).
- `bool` (`true` o `false`).
- `List<T>` (arreglos ordenados).
- `Map<K, V>` (diccionarios o pares clave-valor).
- `Set<T>` (colecciones no duplicadas).

1.4 Ejemplo

A continuación, implementaremos un programa completo ejecutable mediante la CLI de Dart. Este programa demuestra el uso de tipos de datos, Sound Null Safety, interpolación de Strings, colecciones, funciones y control de flujo.

Crear un proyecto Dart->Consola y en el archivo `bin/main.dart`

```
/// Demuestra las características sintácticas principales del lenguaje.

void main() {
  print('=====');
  print('    BIENVENIDOS AL CURSO DART/FLUTTER - ESCOM    ');
  print('=====\\n');

  // 1. Declaración de variables y Sound Null Safety
  final String nombreMateria = 'Desarrollo de Aplicaciones Móviles Nativas';
  const int semestre = 7;

  // Variable anulable (Nullable)
  String? profesorAuxiliar;

  print('Asignatura: $nombreMateria');
  print('Semestre: $semestreº');
  print('Auxiliar asignado: ${profesorAuxiliar ?? "Sin auxiliar asignado"}\\n');

  // 2. Inferencia de tipos y mutabilidad
```

```

var totalAlumnos = 35; // Inferido como int
final double promedioAprobatorio = 8.0; // Inmutable en tiempo de ejecución

print('Total de inscritos: $totalAlumnos');
print('Promedio mínimo para exentar: $promedioAprobatorio\n');

// 3. Manejo de Colecciones (List, Map, Set)
final List<String> temasDart = [
  'Variables y Tipos',
  'Control de Flujo',
  'POO Avanzada',
  'Asincronía (Futures y Streams)'
];

final Map<String, dynamic> perfilEstudiante = {
  'boleta': '2021630000',
  'nombre': 'Ingeniero Politécnico',
  'creditosAprobados': 85.5,
  'esRegular': true,
};

final Set<String> idsLaboratorio = {'LAB-101', 'LAB-102', 'LAB-101'}; // 'LAB-
101' no se duplicará

// 4. Procesamiento de datos con Iteraciones y Métodos de Colección
print('--- Temas a revisar en el bloque de Dart ---');
temasDart.asMap().forEach((index, tema) {
  print('  ${index + 1}. $tema');
});

print('\n--- Perfil del Estudiante Evaluado ---');
perfilEstudiante.forEach((clave, valor) {
  print('  $clave: $valor');
});

print('\nLaboratorios asignados únicos: $idsLaboratorio\n');

// 5. Llamada a funciones puras y control de flujo
final double calificacionEvaluacion = 8.7;
final String resultado = evaluarRendimiento(
  calificacion: calificacionEvaluacion,
  asistencia: 90
);

print('--- Resultado de Evaluación ---');
print('Calificación: $calificacionEvaluacion');
print('Estado: $resultado');
print('=====');
}

/// Función que evalúa el estado del alumno usando parámetros nombrados ({ })
/// y expresión de flecha / control de flujo moderno.
String evaluarRendimiento({required double calificacion, required int asistencia})
{

```

```
if (asistencia < 80) {  
    return 'REPROBADO (Faltas acumuladas)';  
}  
  
// Operador ternario para asignación concisa  
return calificacion >= 6.0  
    ? 'APROBADO'  
    : 'REPROBADO (Calificación insuficiente)';  
}
```

Para ejecutar este código en VS Code:

1. Abran la terminal integrada.
2. Ejecuten el comando:

```
dart run bin/main.dart
```

1.5 Ejercicios Prácticos

Para reforzar lo aprendido, resuelvan los siguientes dos problemas en su entorno local. Deben estructurar su código aplicando las buenas prácticas vistas (tipado estático explícito o inferido correctamente, Sound Null Safety y funciones concisas).

Ejercicio 1: Sistema de Control de Acceso a Laboratorio

Crea un programa en Dart que gestione el acceso de alumnos a los laboratorios de la ESCOM según las siguientes reglas:

1. Define un `Map<String, String?>` donde la clave sea el número de boleta y el valor sea el estado del pase ("PERMITIDO", "DENEGADO", o `null` si no ha sido registrado).
2. Crea una función `verificarAcceso(String boleta, Map<String, String?> registro)` que retorne un mensaje indicando si el alumno puede pasar.
3. Si el estado es `null`, la función debe usar el operador `??` para devolver "ESTADO NO REGISTRADO - REVISAR EN GESTIÓN ESCOLAR".
4. Imprime en consola el procesamiento de al menos 3 alumnos con estados distintos.

Ejercicio 2: Analizador de Calificaciones

Escribe una script que procese una lista de calificaciones enteras (`List<int>`):

1. Filtra las calificaciones reprobatorias (menores a 6) utilizando el método `.where()`.
2. Calcula el promedio general de la clase con el método `.fold()` o un ciclo `for-in`.
3. Retorna un `Set<int>` con las calificaciones únicas obtenidas para eliminar valores duplicados.

4. Muestra en consola el promedio con solo un decimal de precisión (tip: investiga el método `.toFixed()`).
-